INTRODUCTION TO DATA SCIENCE · AUDITORY EXERCISES

Exercise 2 · Part 1: Data and Classical Models

Statistical Inference

Ana Todorovska
Dimitar Peshevski
Marija Stojcheva
Viktor Kostadinoski

Faculty of Computer Science and Engineering
Ss. Cyril and Methodius University in Skopje

Winter Semester
2026/2027

Loading the Data

PYTHON

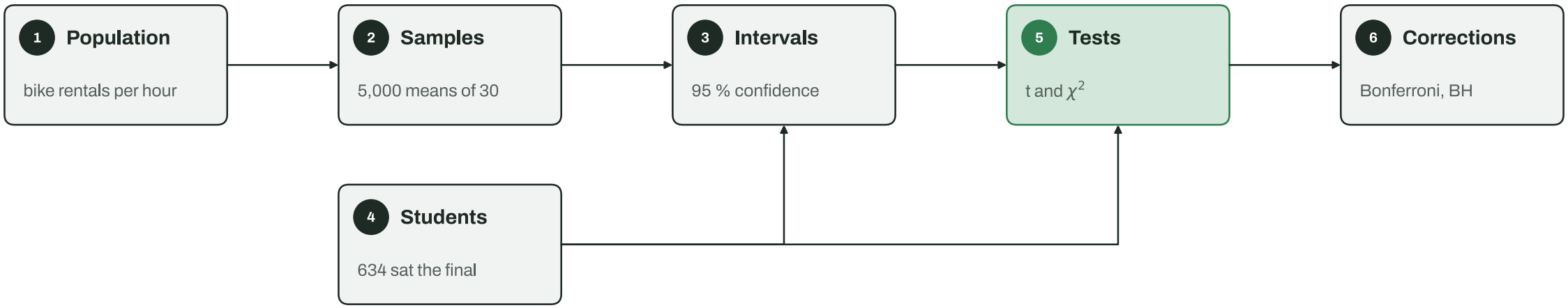
```python
from itertools import combinations

import numpy as np
import pandas as pd
from scipy import stats
from statsmodels.stats.multitest import multipletests

rng = np.random.default_rng(2026)
population = pd.read_csv("data/bike-hours.csv")["rentals"].to_numpy()
students = pd.read_csv("data/student-por.csv", sep=";")
```

The student records separate their fields with semicolons, so `read_csv` takes `sep=";"`.

The Worked Example



The hourly rentals of a bike-sharing system serve as a population whose mean is known. The records of 649 students of the Portuguese language course at two secondary schools in Portugal then answer questions with tests and intervals. Grades run from 0 to 20, and 10 is the pass mark. The highlighted step, the tests, takes most of the session.

01

Sampling and Intervals

Samples, the central limit theorem, intervals, and the bootstrap

Population and Sample

PYTHON

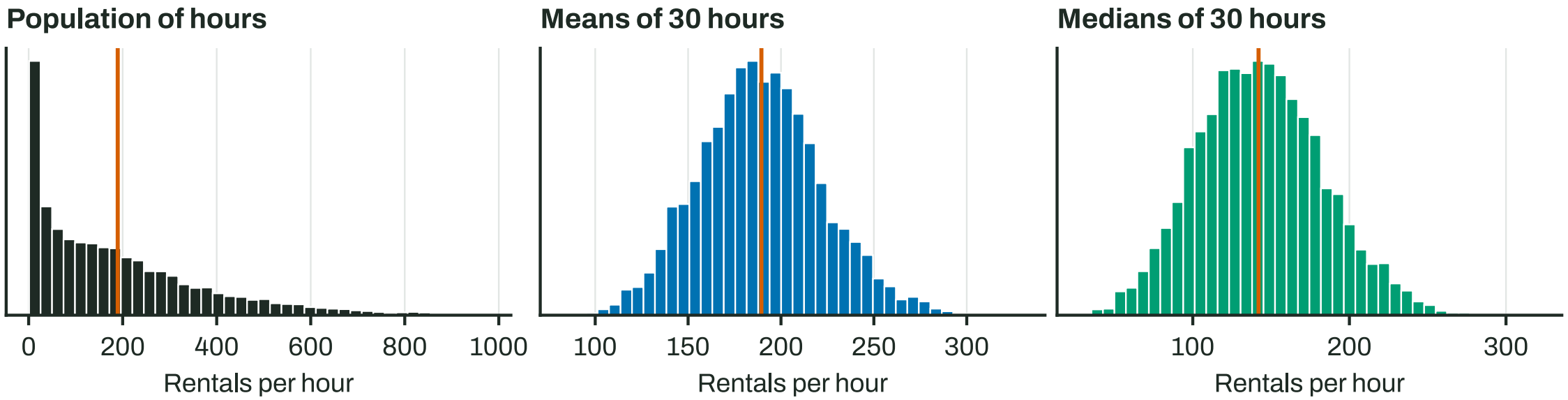
```
mu, sigma = population.mean(), population.std()
sample = rng.choice(population, size=30)
round(mu, 1), round(sample.mean(), 1)
```

OUTPUT

```
(np.float64(189.5), np.float64(171.2))
```

The population is every hour of two years, so its mean μ is known. A sample of 30 hours estimates it, and another sample gives another estimate.

The Sampling Distribution



A statistic computed on 5,000 samples of 30 hours builds its sampling distribution. The means gather around μ in a narrow and nearly symmetric shape, although the population is skewed.

The Standard Error

The Simulation

PYTHON

```
samples = rng.choice(population,
                     size=(5000, 30))
means = samples.mean(axis=1)
round(means.std(), 2)
```

OUTPUT

```
np.float64(32.95)
```

The Formula

PYTHON

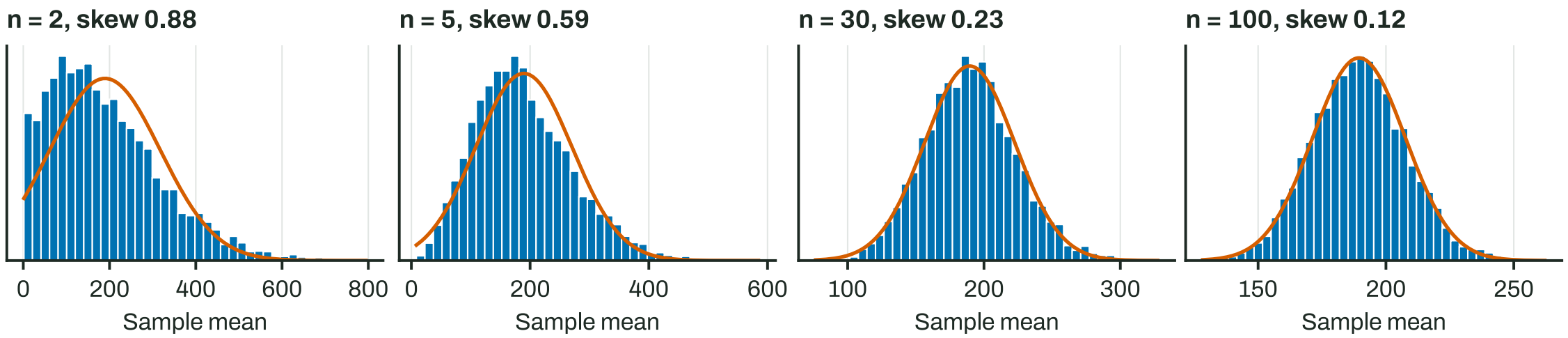
```
round(sigma / np.sqrt(30), 2)
```

OUTPUT

```
np.float64(33.12)
```

The standard deviation of a sampling distribution is the standard error, and for the mean it equals $\sigma / \sqrt{n}$.

The Central Limit Theorem



For independent draws with a finite variance, the sample mean approaches $N(\mu, \sigma^2/n)$ as n grows. A more skewed population needs a larger n before the approximation holds.

A Confidence Interval for a Mean

$$\bar{x} \pm t_{1-\alpha/2,\, n-1}\, \frac{s}{\sqrt{n}}$$

The estimated standard error $s/\sqrt{n}$ replaces the unknown $\sigma/\sqrt{n}$, and the t quantile with $n-1$ degrees of freedom accounts for that estimate. A 95 % interval takes $\alpha = 0.05$.

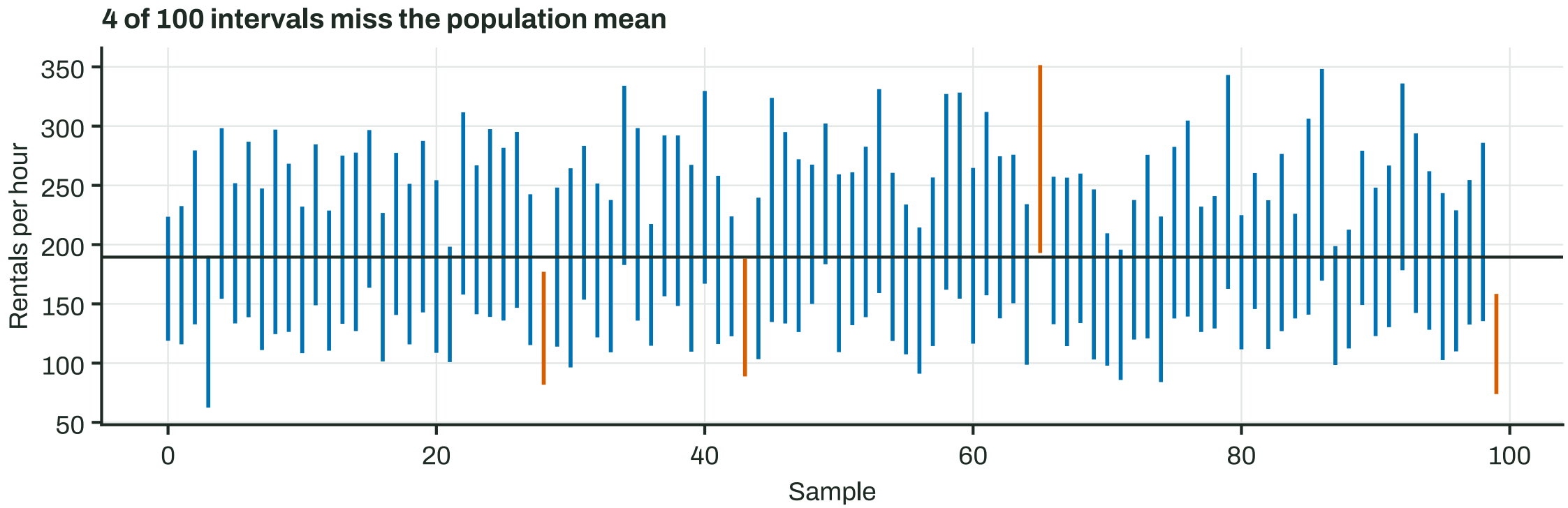
PYTHON

```
np.round(stats.t.interval(0.95, df=29, loc=sample.mean(), scale=stats.sem(sample)), 1)
```

OUTPUT

```
array([118.9, 223.5])
```

What 95 % Means



One interval either contains μ or does not. The 95 % describes the procedure: over many samples, about 95 % of its intervals contain μ. On this skewed population with $n = 30$, the coverage falls slightly short.

Measuring the Coverage

PYTHON

```
half_width = stats.t.ppf(0.975, 29) * samples.std(axis=1, ddof=1) / np.sqrt(30)
(np.abs(means - mu) <= half_width).mean()
```

OUTPUT

```
np.float64(0.9346)
```

The confidence level holds exactly only under the assumptions of the procedure, and a skewed population with a small sample bends it.

NOTEBOOK · SECTIONS 1 TO 3

Samples and Intervals

Run the cells from `## Setup` to `## 3. Confidence Intervals`.

The Students

PYTHON

```
students[["school", "sex", "address", "Mjob", "absences", "G1", "G2", "G3"]].head()
```

	school	sex	address	Mjob	absences	G1	G2	G3
0	GP	F	U	at_home	4	0	11	11
1	GP	F	U	at_home	2	9	11	11
2	GP	F	U	at_home	6	12	13	12
3	GP	F	U	health	0	14	14	14
4	GP	F	U	other	0	11	13	13

`GP` is Gabriel Pereira school, and `MS` is Mousinho da Silveira school. In `address`, `U` means urban and `R` means rural.

Students Who Left Before the Final

PYTHON

```python
left = students[students["G3"] == 0]
len(left), (left["absences"] == 0).mean(), (students["absences"] == 0).mean().round(2)
```

OUTPUT

```
(15, np.float64(1.0), np.float64(0.38))
```

PYTHON

```python
grades = students[students["G3"] > 0].reset_index(drop=True)
g3 = grades["G3"]
```

Every student with a final grade of 0 also has zero recorded absences. These rows describe students who left before the final, so a test of the final grade leaves them out.

An Interval for the Students

PYTHON

```
np.round(stats.t.interval(0.95, df=len(g3) - 1, loc=g3.mean(), scale=stats.sem(g3)), 2)
```

OUTPUT

```
array([11.98, 12.4 ])
```

The interval for the mean final grade follows the same formula as the interval for the mean rentals.

The Bootstrap

PYTHON

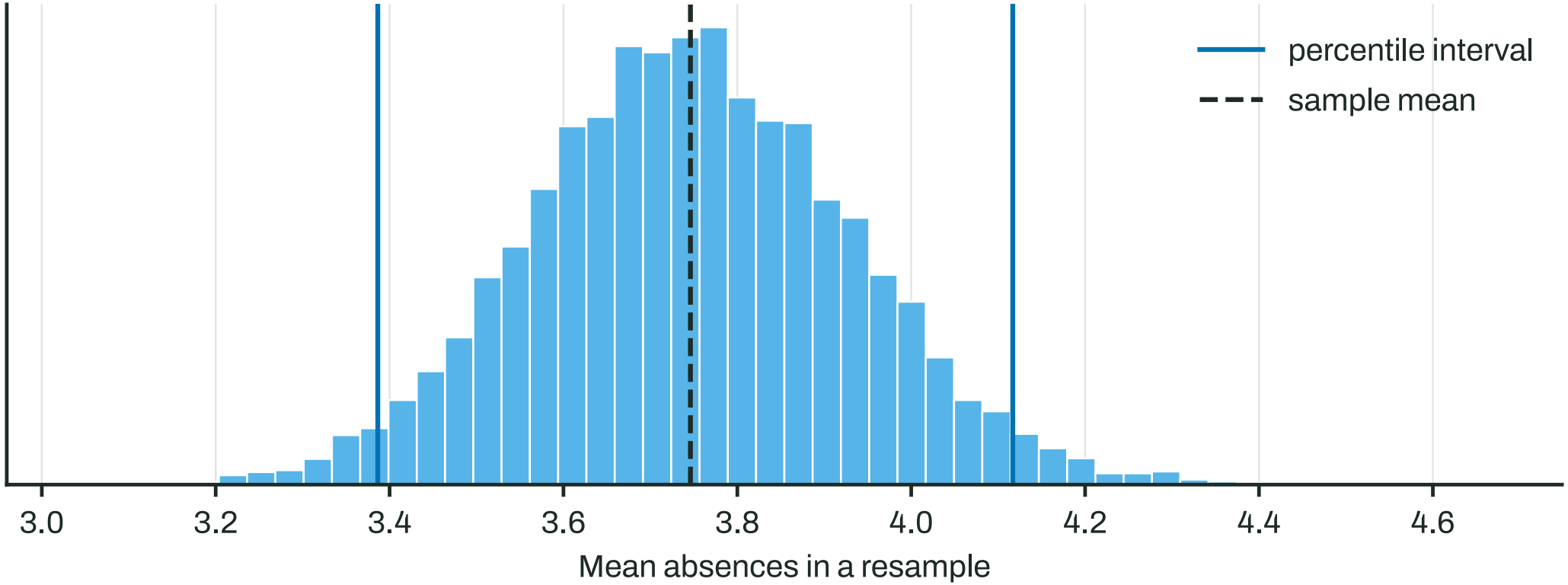
```python
absences = grades["absences"].to_numpy()
percentile = stats.bootstrap((absences,), np.mean, method="percentile", rng=rng)
np.round(percentile.confidence_interval, 2)
```

OUTPUT

```
array([3.4 , 4.11])
```

The bootstrap resamples the observed values with replacement, recomputes the statistic on each resample, and reads the interval from the distribution of the results.

The Percentile Interval



The percentile interval takes the 2.5 % and 97.5 % quantiles of the bootstrap distribution. The absences have a long right tail, but the mean of 634 students is close to normal, so the interval lies close to the t-interval $[3.38, 4.11]$.

What the Bootstrap Cannot Do

PYTHON

```
stats.bootstrap((absences,), np.median, method="percentile", rng=rng).confidence_interval
```

OUTPUT

```
ConfidenceInterval(low=np.float64(2.0), high=np.float64(2.0))
```

The bootstrap only reflects the sample. The median of a column with many repeated values barely moves between resamples, so the interval collapses to a single value, and a sample that misrepresents its population gives an interval that does too.

NOTEBOOK · SECTIONS 4 TO 5

The Students and the Bootstrap

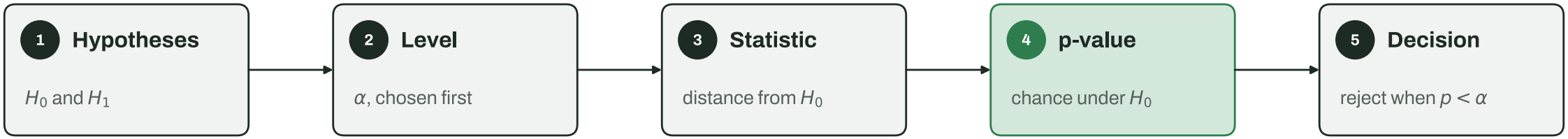
Run the cells from `## 4. The Students` to `## 5. The Bootstrap`.

02

Testing Hypotheses

The p-value, the t-tests, and the chi-squared tests

The Five Steps of a Test

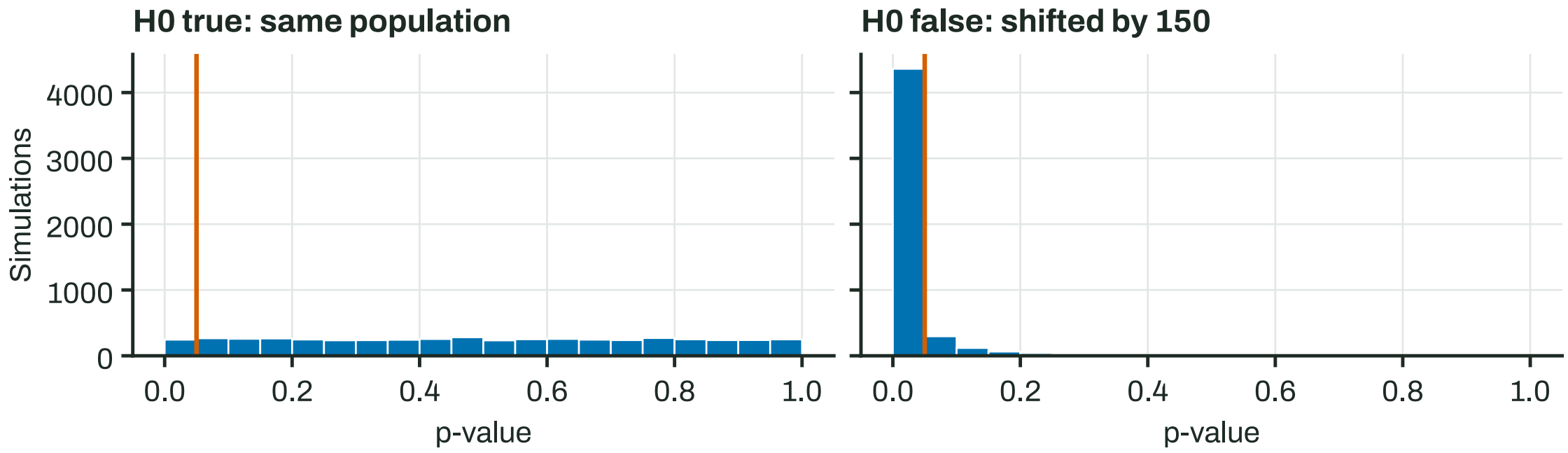


State H_0 and H_1 before looking at the result, choose α, compute a statistic that measures how far the data lie from H_0, compute the p-value, and reject H_0 when the p-value is below α. The highlighted step, the p-value, is what the next slide explains.

What a p-Value Says

Statement	Verdict
The probability under H_0 of a statistic at least as extreme as the observed one.	It is the definition.
The probability that H_0 is true.	It is false, because the p-value assumes H_0.
A large p-value shows that H_0 is true.	It is false, because a small sample also gives a large p-value.
A p-value below α after many tries is evidence.	It is false unless the tries are corrected for.

Two Types of Error



When H_0 is true, p-values spread evenly, and a share α falls below α: these are Type I errors. When H_0 is false, they pile up near 0, and the share above α is the Type II error rate β. The power of the test is $1 - \beta$.

Measuring the Error Rates

PYTHON

```python
first, same = rng.choice(population, size=(2, 5000, 30))
shifted = rng.choice(population, size=(5000, 30)) + 150
p_null = stats.ttest_ind(first, same, axis=1, equal_var=False).pvalue
p_shift = stats.ttest_ind(first, shifted, axis=1, equal_var=False).pvalue
(p_null < 0.05).mean(), (p_shift >= 0.05).mean()
```

OUTPUT

```
(np.float64(0.0446), np.float64(0.119))
```

The Type I error rate lies close to α. The Type II error rate depends on the size of the effect, the spread, and n.

The One-Sample t-Test

PYTHON

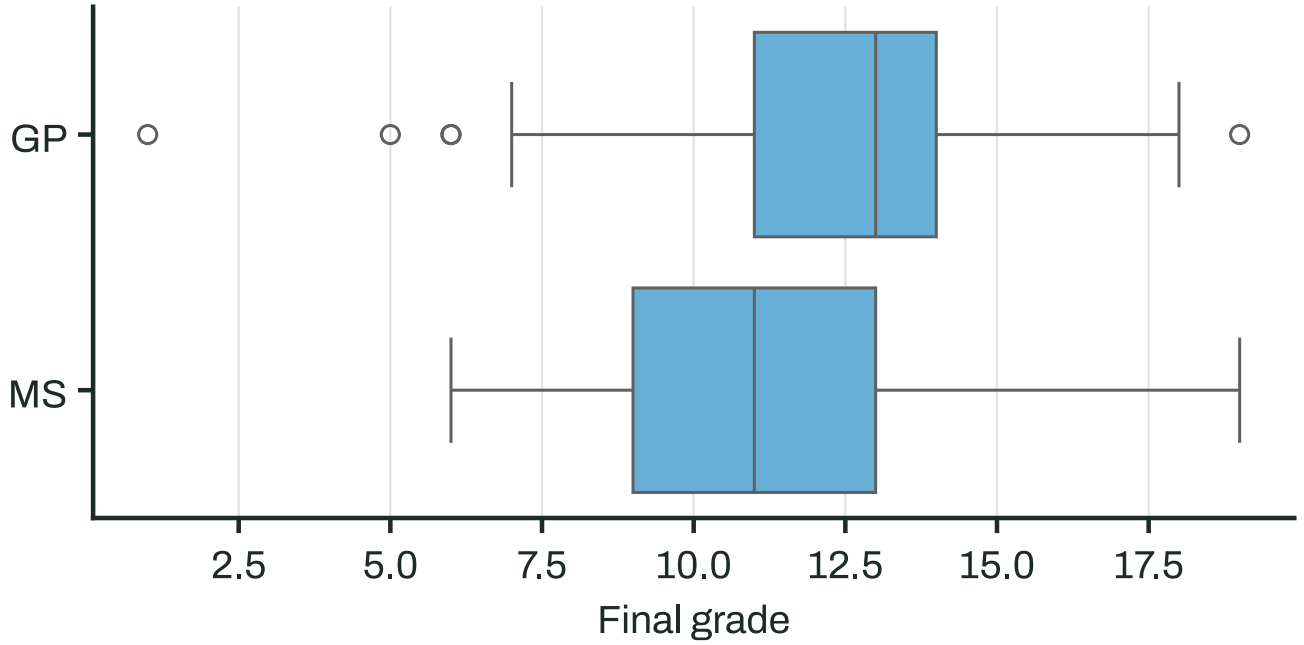
```
stats.ttest_1samp(g3, popmean=10)
```

OUTPUT

```
TtestResult(statistic=np.float64(20.463750023398518),
            pvalue=np.float64(8.072455305526178e-72),
            df=np.int64(633))
```

H_0 states that the mean final grade equals the pass mark of 10. The statistic is $t = (\bar{x} - \mu_0)/(s/\sqrt{n})$ with $n - 1$ degrees of freedom.

Welch's Two-Sample t-Test



PYTHON

```python
gp = g3[grades["school"] == "GP"]
ms = g3[grades["school"] == "MS"]
welch = stats.ttest_ind(gp, ms,
                        equal_var=False)
low, high = welch.confidence_interval()
(f"p = {welch.pvalue:.1e}, "
 f"[{low:.2f}, {high:.2f}]")
```

OUTPUT

```
'p = 6.3e-08, [0.81, 1.70]'
```

Welch's test compares two independent means without assuming equal variances. It rejects H_0, and the interval puts the difference of the school means between 0.81 and 1.70 grade points.

NOTEBOOK · SECTIONS 6 TO 7

Testing Means

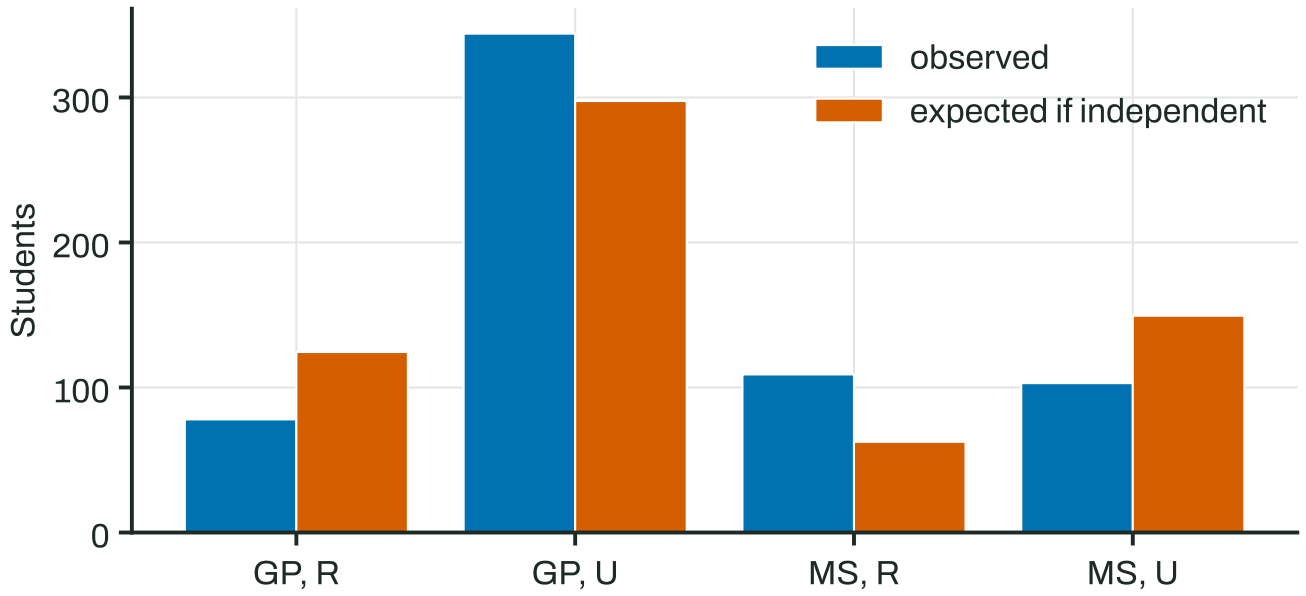
Run the cells from `## 6. Reading a p-Value` to `## 7. The t-Tests`.

The Chi-Squared Statistic

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$$

O_i is an observed count and E_i is the count expected under H_0. For a table of independence, $E = \text{row total} \times \text{column total}/n$, with $(r-1)(c-1)$ degrees of freedom. The approximation needs every E_i to be at least 5.

The Test of Independence



PYTHON

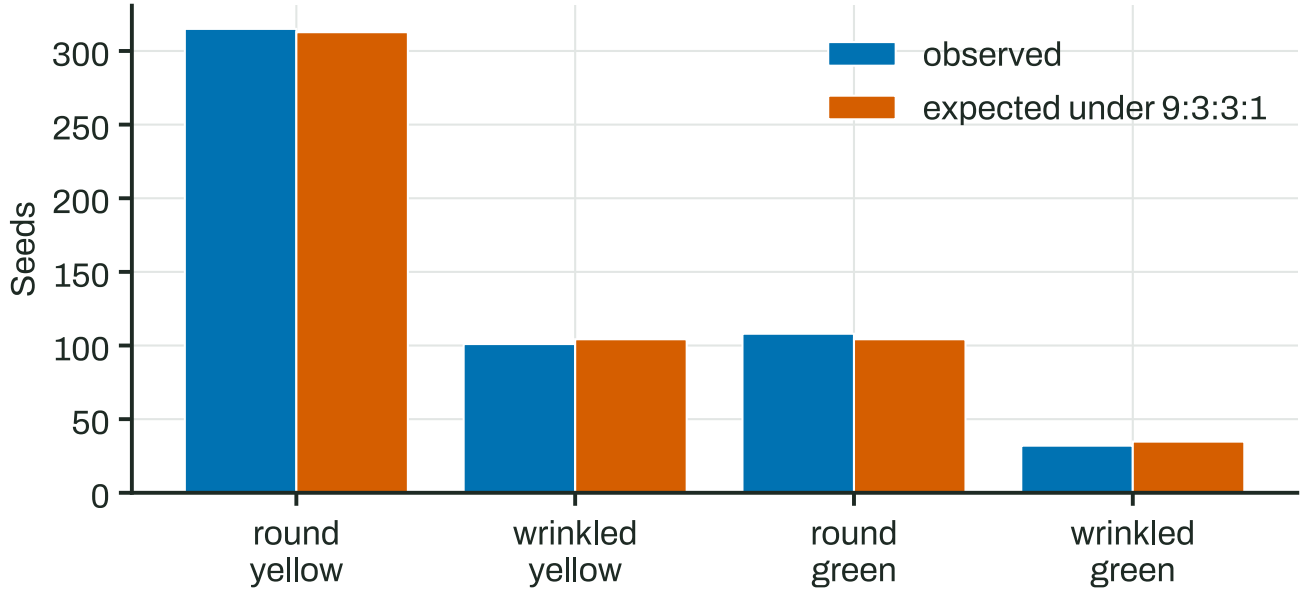
```
table = pd.crosstab(grades["school"],
                    grades["address"])
result = stats.chi2_contingency(table)
(f"p = {result.pvalue:.1e}, "
 f"min E = {result.expected_freq.min():.1f}")
```

OUTPUT

```
'p = 2.1e-17, min E = 62.5'
```

The schools draw on different areas. The smallest expected count E is well above 5, so the approximation holds.

The Test of Goodness of Fit



PYTHON

```
mendel = pd.read_csv("data/mendel.csv")
cross = mendel[mendel["cross"]
               == "shape and colour"]
expected = (cross["ratio"]
            / cross["ratio"].sum()
            * cross["count"].sum())
stats.chisquare(cross["count"],
                f_exp=expected)
```

OUTPUT

```
Power_divergenceResult(
  statistic=np.float64(0.4700239808153477),
  pvalue=np.float64(0.925425895103616))
```

Mendel's two-trait cross fits the ratio his model predicts, so the data do not contradict the model.

NOTEBOOK · SECTION 8

Counts

Run the cells of `## 8. The Chi-Squared Tests`.

03

Many Tests at Once

The family-wise error rate, Bonferroni, and Benjamini-Hochberg

The Family-Wise Error Rate

$$P(\text{at least one false rejection}) = 1 - (1 - \alpha)^m$$

With $\alpha = 0.05$ and $m = 10$ independent true null hypotheses, the chance of at least one false rejection is 0.40. Welch's test of every pair of the five occupations of the mother in `Mjob` gives ten tests. They share their groups, so they are not independent, but they form a family of the same size.

PYTHON

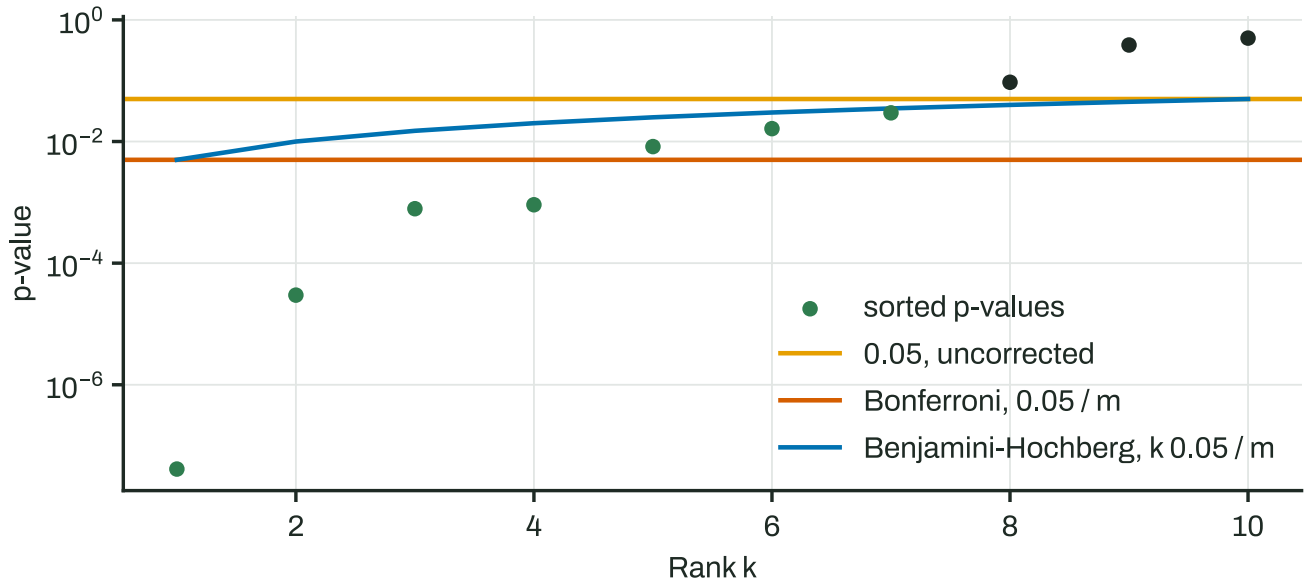
```python
pairs = list(combinations(sorted(grades["Mjob"].unique()), 2))
job = grades["Mjob"]
p = [stats.ttest_ind(g3[job == a], g3[job == b], equal_var=False).pvalue for a, b in pairs]
```

Two Corrections

Bonferroni. Reject $H_{(k)}$ when $p_{(k)} < \alpha/m$. It holds the family-wise error rate, the chance of any false rejection, at α.

Benjamini-Hochberg. Find the largest k with $p_{(k)} \leq k\alpha/m$, and reject the k smallest p-values. It holds the false discovery rate, the expected share of false rejections among the rejections, at α, and keeps more power.

Corrections in Use



PYTHON

```
p = np.array(p)
bonferroni = multipletests(
    p, method="bonferroni")[0]
bh = multipletests(
    p, method="fdr_bh")[0]
(f"{(p < 0.05).sum()} raw, "
 f"{bonferroni.sum()} Bonferroni, "
 f"{bh.sum()} BH")
```

OUTPUT

```
'7 raw, 4 Bonferroni, 7 BH'
```

Bonferroni suits a family in which any false claim is costly. Benjamini-Hochberg suits a screen whose discoveries are checked afterwards.

NOTEBOOK · SECTION 9

Correcting the Pairwise Tests

Run the cells of `## 9. Multiple Comparisons`.



Reference

Variants, alternatives, and parameters for the tasks of this part

Intervals for a Mean

Option	When It Is the Right Choice
`stats.t.interval(level, df, loc, scale)`	It builds the interval from a mean and a standard error that are already computed.
`stats.ttest_1samp(x, m).confidence_interval(confidence_level=)`	It gives the interval of the mean together with the test.
`stats.sem(x)`	It computes the estimated standard error $s/\sqrt{n}$.

Bootstrap Parameters

Option	When It Is the Right Choice
`method="percentile"`	It takes the quantiles of the bootstrap distribution, and it is passed explicitly because the default of `stats.bootstrap` is another method.
`n_resamples=9999`	It is the default number of resamples, and more resamples only reduce the simulation noise.
`confidence_level=0.95`	It sets the level of the interval.
`rng=`	It fixes the generator, so that a rerun gives the same interval.

The t-Tests

Option	When It Is the Right Choice
`stats.ttest_1samp(x, popmean=)`	It compares one mean with a fixed value.
`stats.ttest_ind(a, b, equal_var=False)`	It is Welch's test for two independent groups, and `equal_var=False` is passed because the default assumes equal variances.
`.confidence_interval(confidence_level=)`	It gives the interval of the mean or of the difference.

The Chi-Squared Tests

Option	When It Is the Right Choice
`stats.chi2_contingency(t)`	It tests independence and returns the expected counts in `.expected_freq`.
`stats.chisquare(f_obs, f_exp=)`	It tests counts against expected counts with the same total.

Corrections

Option	When It Is the Right Choice
`multipletests(p, method="bonferroni")`	It holds the family-wise error rate when any false rejection is costly.
`multipletests(p, method="fdr_bh")`	It holds the false discovery rate for a screen of many tests.
`multipletests(...)[1]`	It gives the corrected p-values, which are compared with α directly.
`stats.false_discovery_control(p)`	It gives the Benjamini-Hochberg corrected p-values in SciPy.

Corrected p-Values

PYTHON

```
pd.DataFrame({"p": p, "bonferroni": multipletests(p, method="bonferroni")[1],
              "bh": stats.false_discovery_control(p)},
             index=[f"{a} - {b}" for a, b in pairs]).sort_values("p").round(4).head(7)
```

OUTPUT

	p	bonferroni	bh
at_home - teacher	0.0000	0.0000	0.0000
other - teacher	0.0000	0.0003	0.0001
at_home - health	0.0008	0.0079	0.0023
services - teacher	0.0009	0.0091	0.0023
at_home - services	0.0083	0.0825	0.0165
at_home - other	0.0163	0.1629	0.0271
health - other	0.0296	0.2961	0.0423

Choosing a Test for Groups

Question and Design	Test
One mean against a fixed value.	`ttest_1samp`, or the bootstrap for another statistic.
Two independent means.	`ttest_ind(equal_var=False)`.
Every pair of several independent means.	`ttest_ind(equal_var=False)` for each pair, then a correction.

Choosing a Test for Counts

Question and Design	Test
Two categorical variables.	`chi2_contingency`.
Counts against a model.	`chisquare` with `f_exp`.

Assumptions

Test	It Assumes
Every test	The observations are independent.
The t-tests	The means are approximately normal, which a large sample or a mild skew grants.
The chi-squared tests	Every expected count is at least 5.